

UNIT18: SKINNED MESH - GEOMETRY

【学習要項】

- ☐Scene
- ☐Node
- ☐Mesh

【演習手順】

1. シェーダーファイルの追加

①HLSL ヘッダーファイル(skinned_mesh.hlsl)

```
1: struct VS_IN
2: {
3:     float4 position : POSITION;
4:     float4 normal : NORMAL;
5:     float2 texcoord : TEXCOORD;
6: };
7: struct VS_OUT
8: {
9:     float4 position : SV_POSITION;
10:    float4 world_position : POSITION;
11:    float4 world_normal : NORMAL;
12:    float2 texcoord : TEXCOORD;
13:    float4 color : COLOR;
14: };
15: cbuffer OBJECT_CONSTANT_BUFFER : register(b0)
16: {
17:     row_major float4x4 world;
18:     float4 material_color;
19: };
20: cbuffer SCENE_CONSTANT_BUFFER : register(b1)
21: {
22:     row_major float4x4 view_projection;
23:     float4 light_direction;
24:     float4 camera_position;
25: };
```

②頂点シェーダーファイル(skinned_mesh_vs.hlsl)

```
1: #include "skinned_mesh.hlsl"
2: VS_OUT main(VS_IN vin)
3: {
4:     VS_OUT vout;
5:     vout.position = mul(vin.position, mul(world, view_projection));
6:     vout.world_position = mul(vin.position, world);
7:     vin.normal.w = 0;
8:     vout.world_normal = normalize(mul(vin.normal, world));
9:     vout.texcoord = vin.texcoord;
10:    vout.color = material_color;
11:    return vout;
12: }
```

③ピクセルシェーダーファイル(skinned_mesh_ps.hlsl)

```
1: #include "skinned_mesh.hlsl"
2: float4 main(VS_OUT pin) : SV_TARGET
3: {
4:     float3 N = normalize(pin.world_normal.xyz);
5:     float3 L = normalize(-light_direction.xyz);
6:     float3 diffuse = max(0, dot(N, L));
7:     return float4(diffuse, 1) * pin.color;
8: }
```

2. skinned_mesh クラスの実装

①skinned_mesh クラスにメッシュ構造体を定義する

```
1: struct mesh
```

```
2: {
3:     uint64_t unique_id{ 0 };
4:     std::string name;
5:     // 'node_index' is an index that refers to the node array of the scene.
6:     int64_t node_index{ 0 };
7:
8:     std::vector<vertex> vertices;
9:     std::vector<uint32_t> indices;
10:
11: private:
12:     Microsoft::WRL::ComPtr<ID3D11Buffer> vertex_buffer;
13:     Microsoft::WRL::ComPtr<ID3D11Buffer> index_buffer;
14:     friend class skinned_mesh;
15: };
16: std::vector<mesh> meshes;
```

② skinned_mesh クラスに fetch_meshes メンバ関数を定義・実装する

```
1: void skinned_mesh::fetch_meshes(FbxScene* fbx_scene, std::vector<mesh>& meshes)
2: {
3:     for (const scene::node& node : scene_view.nodes)
4:     {
5:         if (node.attribute != FbxNodeAttribute::EType::eMesh)
6:         {
7:             continue;
8:         }
9:
10:        FbxNode* fbx_node{ fbx_scene->FindNodeByName(node.name.c_str()) };
11:        FbxMesh* fbx_mesh{ fbx_node->GetMesh() };
12:
13:        mesh& mesh{ meshes.emplace_back() };
14:        mesh.unique_id = fbx_node->GetUniqueID();
15:        mesh.name = fbx_node->GetName();
16:        mesh.node_index = scene_view.indexof(mesh.unique_id);
17:
18:        const int polygon_count{ fbx_mesh->GetPolygonCount() };
19:        mesh.vertices.resize(polygon_count * 3LL);
20:        mesh.indices.resize(polygon_count * 3LL);
21:
22:        FbxStringList uv_names;
23:        fbx_mesh->GetUVSetNames(uv_names);
24:        const FbxVector4* control_points{ fbx_mesh->GetControlPoints() };
25:        for (int polygon_index = 0; polygon_index < polygon_count; ++polygon_index)
26:        {
27:            for (int position_in_polygon = 0; position_in_polygon < 3; ++position_in_polygon)
28:            {
29:                const int vertex_index{ polygon_index * 3 + position_in_polygon };
30:
31:                vertex vertex;
32:                const int polygon_vertex{ fbx_mesh->GetPolygonVertex(polygon_index, position_in_polygon) };
33:                vertex.position.x = static_cast<float>(control_points[polygon_vertex][0]);
34:                vertex.position.y = static_cast<float>(control_points[polygon_vertex][1]);
35:                vertex.position.z = static_cast<float>(control_points[polygon_vertex][2]);
36:
37:                if (fbx_mesh->GetElementNormalCount() > 0)
38:                {
39:                    FbxVector4 normal;
40:                    fbx_mesh->GetPolygonVertexNormal(polygon_index, position_in_polygon, normal);
41:                    vertex.normal.x = static_cast<float>(normal[0]);
42:                    vertex.normal.y = static_cast<float>(normal[1]);
43:                    vertex.normal.z = static_cast<float>(normal[2]);
44:                }
45:                if (fbx_mesh->GetElementUVCount() > 0)
46:                {
47:                    FbxVector2 uv;
```

UNIT18: SKINNED MESH - GEOMETRY

```
48:         bool unmapped_uv;
49:         fbx_mesh->GetPolygonVertexUV(polygon_index, position_in_polygon,
50:             uv_names[0], uv, unmapped_uv);
51:         vertex.texcoord.x = static_cast<float>(uv[0]);
52:         vertex.texcoord.y = 1.0f - static_cast<float>(uv[1]);
53:     }
54:
55:     mesh.vertices.at(vertex_index) = std::move(vertex);
56:     mesh.indices.at(vertex_index) = vertex_index;
57: }
58:
59: }
60: }
61: }
```

③skinned_mesh クラスに create_com_objects メンバ関数を定義・実装する

```
1: void skinned_mesh::create_com_objects(ID3D11Device* device, const char* fbx_filename)
2: {
3:     for (mesh& mesh : meshes)
4:     {
5:         HRESULT hr{ S_OK };
6:         D3D11_BUFFER_DESC buffer_desc{};
7:         D3D11_SUBRESOURCE_DATA subresource_data{};
8:         buffer_desc.ByteWidth = static_cast<UINT>(sizeof(vertex) * mesh.vertices.size());
9:         buffer_desc.Usage = D3D11_USAGE_DEFAULT;
10:        buffer_desc.BindFlags = D3D11_BIND_VERTEX_BUFFER;
11:        buffer_desc.CPUAccessFlags = 0;
12:        buffer_desc.MiscFlags = 0;
13:        buffer_desc.StructureByteStride = 0;
14:        subresource_data.pSysMem = mesh.vertices.data();
15:        subresource_data.SysMemPitch = 0;
16:        subresource_data.SysMemSlicePitch = 0;
17:        hr = device->CreateBuffer(&buffer_desc, &subresource_data,
18:            mesh.vertex_buffer.ReleaseAndGetAddressOf());
19:        _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
20:
21:        buffer_desc.ByteWidth = static_cast<UINT>(sizeof(uint32_t) * mesh.indices.size());
22:        buffer_desc.Usage = D3D11_USAGE_DEFAULT;
23:        buffer_desc.BindFlags = D3D11_BIND_INDEX_BUFFER;
24:        subresource_data.pSysMem = mesh.indices.data();
25:        hr = device->CreateBuffer(&buffer_desc, &subresource_data,
26:            mesh.index_buffer.ReleaseAndGetAddressOf());
27:        _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
28:        #if 1
29:            mesh.vertices.clear();
30:            mesh.indices.clear();
31:        #endif
32:    }
33:
34:    HRESULT hr = S_OK;
35:    D3D11_INPUT_ELEMENT_DESC input_element_desc[]
36:    {
37:        { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, D3D11_APPEND_ALIGNED_ELEMENT },
38:        { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, D3D11_APPEND_ALIGNED_ELEMENT },
39:        { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, D3D11_APPEND_ALIGNED_ELEMENT },
40:    };
41:    create_vs_from_cso(device, "skinned_mesh_vs.cso", vertex_shader.ReleaseAndGetAddressOf(),
42:        input_layout.ReleaseAndGetAddressOf(), input_element_desc, ARRAYSIZE(input_element_desc));
43:    create_ps_from_cso(device, "skinned_mesh_ps.cso", pixel_shader.ReleaseAndGetAddressOf());
44:
45:    D3D11_BUFFER_DESC buffer_desc{};
46:    buffer_desc.ByteWidth = sizeof(constants);
47:    buffer_desc.Usage = D3D11_USAGE_DEFAULT;
48:    buffer_desc.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
```

UNIT18: SKINNED MESH - GEOMETRY

```
49:     hr = device->CreateBuffer(&buffer_desc, nullptr, constant_buffer.ReleaseAndGetAddressOf());
50:     _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
51: }
```

④skinned_mesh コンストラクタのコードを変更する

```
1:  skinned_mesh::skinned_mesh(ID3D11Device* device, const char* fbx_filename, bool triangulate)
2:  {
3:
4:      :
5:      :
6:      :
7:
8:      traverse(fbx_scene->GetRootNode());
9:
10:     fetch_meshes(fbx_scene, meshes);
11:
12:     fbx_manager->Destroy();
13:
14:     create_com_objects(device, fbx_filename);
15: }
```

⑤skinned_mesh クラスに render メンバ関数を定義・実装する

```
1:  void skinned_mesh::render(ID3D11DeviceContext* immediate_context,
2:      const XMFLOAT4X4& world, const XMFLOAT4& material_color)
3:  {
4:      for (const mesh& mesh : meshes)
5:      {
6:          uint32_t stride{ sizeof(vertex) };
7:          uint32_t offset{ 0 };
8:          immediate_context->IASetVertexBuffers(0, 1, mesh.vertex_buffer.GetAddressOf(), &stride, &offset);
9:          immediate_context->IASetIndexBuffer(mesh.index_buffer.Get(), DXGI_FORMAT_R32_UINT, 0);
10:         immediate_context->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
11:         immediate_context->IASetInputLayout(input_layout.Get());
12:
13:         immediate_context->VSSetShader(vertex_shader.Get(), nullptr, 0);
14:         immediate_context->PSSetShader(pixel_shader.Get(), nullptr, 0);
15:
16:         constants data;
17:         data.world = world;
18:         data.material_color = material_color;
19:         immediate_context->UpdateSubresource(constant_buffer.Get(), 0, 0, &data, 0, 0);
20:         immediate_context->VSSetConstantBuffers(0, 1, constant_buffer.GetAddressOf());
21:
22:         D3D11_BUFFER_DESC buffer_desc;
23:         mesh.index_buffer->GetDesc(&buffer_desc);
24:         immediate_context->DrawIndexed(buffer_desc.ByteWidth / sizeof(uint32_t), 0, 0);
25:     }
26: }
```

3. framework クラスの render メンバ関数で skinned_mesh クラスの render メンバ関数を呼び出す

- ①拡大縮小 (S)・回転 (R)・平行移動 (T) 行列を計算する
- ②上記 3 行列を合成しワールド変換行列 (world) を作成する
- ③skinned_mesh クラスの render メンバ関数を呼び出す

```
skinned_meshes[0]->render(immediate_context.Get(), world, { 1, 0, 0, 1 });
```

4. 実行し、正立方体が描画される事を確認する

【評価項目】

- ☐頂点位置・法線情報のみを持つ FBX ファイルのロードと描画
- ☐描画色の変更